

SpecDrift-Bench: A Fault-Injection Benchmark for Evaluating LLM-Based Specification-Drift Detection

K Nandesh
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
127018025@sastra.ac.in

S Soundar
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
127018056@sastra.ac.in

N Sudharsan
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
127018059@sastra.ac.in

Vivekanandan S J
School of Computing
SASTRA Deemed to be
University
Thanjavur, India
vivekanandansj@sastra.ac.in

Abstract—Large language models now write a substantial share of production code, and they introduce a failure mode that functional testing does not catch: specification drift, in which an implementation quietly stops satisfying a requirement it once met while the project's own test suite continues to pass. Tools that claim to detect drift are evaluated by the issues they report rather than by the issues they miss or invent, so their trustworthiness is unknown. This paper presents SpecDrift-Bench, a fault-injection benchmark and evaluation framework that treats drift detection as a classification problem in which false alarms are weighted as heavily as misses. We define nine drift categories, including comment decoys and reverse-direction unauthorised scope creep, together with three categories of negative control, and construct 86 validated cases by injecting faults into three working Python projects governed by 40 numbered specification rules. Every case records whether the host project's own tests still pass: 22 percent of injected drift escapes testing entirely, and unauthorised scope creep escapes it in every case. We evaluate a two-stage detector, SpecGuard, whose retrieval and verification stages are instrumented separately so that each miss is attributed to one of them, against keyword-matching and whole-file-prompting baselines. Retrieval raises recall from 0.17 to 0.58 but raises the false-alarm rate from 0.04 to 0.63, and the surviving false alarms are confidently wrong rather than unsupported. We conclude that evidence citation alone cannot make an LLM conformance judge trustworthy and that its counterexamples must be executed.

Keywords—specification drift, benchmark construction, fault injection, LLM-based verification, false-positive analysis, empirical software engineering

I. INTRODUCTION

Large language models have moved from autocompletion to authorship. Code is now generated, refactored and extended by assistants that read a natural-language requirement once and then operate on the repository for weeks afterwards. The requirement does not travel with the code. What results is specification drift: the implementation diverges from the requirement that justified it, and nothing in the ordinary development loop announces the divergence.

Drift is not the same as a bug. A bug breaks behaviour the tests describe, and the tests go red. Drift breaks behaviour the specification describes but the tests happen not to cover, so the suite stays green. The gap is not hypothetical. In the benchmark presented here, 22 percent of all injected drift leaves the host project's own test suite passing, and one category of drift, behaviour added beyond what the specification authorises, survives the tests in every single case. The tests were written before that behaviour existed, so they cannot fail on it.

Three families of tooling address the problem and none of them measures its own reliability. Static analysers and keyword-based conformance checks match vocabulary rather than behaviour; a rule that forbids a fourth concurrent loan is not satisfied or violated by the presence of the word "loan". Spec-driven development toolkits such as GitHub Spec Kit and AWS Kiro validate adherence at generation time and never again, which leaves every subsequent edit unchecked. LLM-based conformance checkers such as Semcheck do verify continuously, but their published evaluations report how many issues were raised, not how many real violations were missed and how many reported issues were fabrications. Without the second number a drift detector cannot be deployed: a checker that flags two thirds of untouched code is noise, however many genuine violations it also catches.

There is good reason to expect that second number to be bad. Jin and Chen [1] show that when LLMs are asked to judge whether correct code conforms to its stated requirement, they systematically overcorrect, declaring compliant implementations defective at high rates, and that the rate rises sharply when the prompt asks the model to explain itself and propose a correction. Their study establishes the failure mode on function-level generation benchmarks, where the requirement is a docstring and the code is a single function. The present work asks what happens when the same judgement is made at repository scale, against a numbered specification document, on code the model must first locate for itself.

This paper makes five contributions.

- A taxonomy of specification drift with nine drift categories and three categories of negative control, including comment decoys, in which the violating code is deleted and a comment asserting the rule is left behind, and reverse-direction unauthorised scope creep, in which no rule is violated because no rule authorises the behaviour at all.
- SpecDrift-Bench, an open benchmark of 86 validated cases built by injecting faults into three working Python projects governed by 40 numbered rules, in which every case is machine-validated and labelled with whether it escapes the host project's tests.
- SpecGuard, a two-stage detector whose rule-retrieval and semantic-verification stages are instrumented independently, so that every miss is attributed either to the code never being shown or to the model reading it wrongly.
- An evaluation protocol in which a drift flag counts only when it names the rule the injection targeted, and in

which every flag on a negative control is a false alarm weighted as heavily as a miss.

- An empirical result with an uncomfortable shape: retrieval more than triples recall and simultaneously multiplies the false-alarm rate by fifteen, and the false alarms that survive an evidence-citation requirement are not unsupported guesses but confidently argued misreadings.

II. BACKGROUND AND RELATED WORK

A. Specification Drift in AI-Assisted Development

Requirements engineering has long distinguished verification, which asks whether the system was built right, from validation, which asks whether the right system was built [10]. Automated testing addresses the first question against an executable encoding of the requirement. Specification drift lives in the residue: requirements that were stated in prose, implemented once, and never encoded as a test. Before LLM-assisted development this residue decayed slowly, because each edit was made by a developer who had at least read the surrounding code. An assistant that rewrites a function to satisfy a local instruction has no such context, and the decay is no longer slow.

B. One-Time Spec-Driven Toolkits

GitHub Spec Kit [3] and AWS Kiro [4] formalise the specification as a first-class artefact and drive generation from it. Both check adherence when the code is produced. Neither re-checks afterwards, so they constrain the first commit and nothing after it. The drift this paper measures is precisely what accumulates after that first commit.

C. Continuous LLM Conformance Checkers

Semcheck [2] represents the continuous alternative: a specification and an implementation are handed to an LLM, which reports semantic mismatches, and the check runs in continuous integration. The design is right and the evaluation is the problem. Reported results take the form of issue counts on real repositories. An issue count has no denominator. It cannot distinguish a checker that finds nine violations out of ten from one that finds nine out of ninety, and it says nothing at all about how often the checker reports a violation in code that is correct. SpecDrift-Bench exists to supply both denominators.

D. Reliability of LLM Conformance Judgement

The base study for this work is Jin and Chen [1], who evaluate five representative LLMs on requirement conformance judgement over HumanEval and MBPP and report systematic overcorrection. GPT-4o reaches a false negative rate of 26.2 percent on HumanEval and 35.9 percent on MBPP under a plain verdict prompt, meaning that it rejects correct implementations at those rates. The striking result is what happens when the prompt is enriched: requiring the model to explain its verdict and propose a repair raises the HumanEval false negative rate to 73.2 percent. Prompt elaboration, normally a reliability intervention, makes the judge markedly worse. The authors trace most false rejections to unsupported "logic error" assertions and propose a Fix-guided Verification Filter that treats the model's proposed fix as executable counterfactual evidence, validating the original and revised programs against the benchmark's reference tests and a specification-constrained augmented suite.

Three design decisions in SpecGuard follow directly from that study. The verification prompt never asks for a fix. A drift verdict is accepted only if it quotes the clause it claims is

violated and supplies a concrete counterexample. And a possible risk is stated, in the prompt itself, not to be a violation. Our contribution is complementary rather than competing: Jin and Chen characterise the failure on function-level generation benchmarks and mitigate it with execution, whereas we measure whether it persists at repository scale, per drift category, once the model must also locate the code it is judging.

E. Code Generation Benchmarks

HumanEval [5], MBPP [6] and the corrected EvalPlus suite [7] measure whether generated code is functionally correct against hidden tests. SWE-bench [11] measures whether an agent can resolve a real issue. All three measure production. None measures surveillance: whether a tool notices, after the fact, that existing code has stopped matching an existing written requirement. That is the gap SpecDrift-Bench occupies.

III. BENCHMARK DESIGN

A. Host Projects and Specifications

A drift benchmark needs code whose requirements are written down, atomic and testable. We therefore authored three small but complete Python services rather than mining repositories, because mined code has no trustworthy rule-level ground truth. Each project ships a specification document of numbered rules, a source tree and a test suite with one test per rule, and each project passes its own suite before any injection. Table I summarises them.

TABLE I. HOST PROJECTS AND THEIR SPECIFICATIONS

Project	Domain	Rules	Cases
library	Loans, catalogue, late fees	14	28
wallet	Transfers, balances, fees	13	29
ratelimiter	Token buckets, throttling	13	29
Total	—	40	86

A representative rule reads: R06, a member **MUST NOT** hold more than 3 active loans at once. Rules are deliberately atomic, so that a verdict on one rule is not entangled with a verdict on another, and deliberately quantified, so that a violation has a witness.

B. A Taxonomy of Drift

Nine drift categories and three control categories are defined in Table II. Eight drift categories are forward: existing behaviour is changed so that it no longer satisfies a rule. One, D8, is reverse: behaviour is added that no rule authorises, such as an undocumented loyalty discount applied inside a transfer. Reverse drift is the category existing tooling is structurally unable to see, because there is no rule to check the new code against, and it is the category that defeats testing most completely.

TABLE II. DRIFT AND CONTROL TAXONOMY

ID	Category	Direction	n
D1	Boundary shift	forward	8
D2	Value / constant drift	forward	9
D3	Omitted check	forward	6
D4	Weakened condition	forward	6
D5	Sequence violation	forward	6
D6	Error-handling drift	forward	6
D7	Comment decoy	forward	6
D8	Unauthorised scope creep	reverse	6
D9	Output contract drift	forward	6
N1	Unchanged code	control	3
N2	Semantics-preserving refactor	control	18
N3	Benign decoy	control	6

The negative controls carry as much of the design weight as the drift cases. N2, semantics-preserving refactoring, is the largest single category in the benchmark at 18 cases, because it is the condition under which a detector is most likely to hallucinate a violation: the code has visibly changed and nothing about its behaviour has. N3 inverts D7, adding a comment that mentions a rule without touching the code that implements it.

C. Case Construction and Validation

Each case is a declarative YAML entry naming the host project, the category, the ground-truth label, the rules the injection targets, the file, a find pattern, a replace pattern, and the qualified name of the gold code symbol the detector would have to retrieve. The builder copies the host project into an isolated workspace and applies the substitution, which must match exactly once; zero matches or two matches fail the build rather than producing a silently wrong case. Host projects are never modified in place.

The validator then compiles the patched file and runs the host project's own test suite inside the workspace. A negative control whose tests fail is marked invalid and excluded from scoring, on the reasoning that a refactor which changes behaviour is not a refactor, and scoring it would corrupt the false-alarm rate. All 86 cases in the present release validate successfully.

D. Test Escape as a First-Class Label

Every drift case records whether the host project's suite still passes after the injection. This turns the paper's motivating claim into a measurement rather than an assertion. Table III gives the result.

TABLE III. DRIFT THAT SURVIVES THE HOST PROJECT'S OWN TESTS

Category	n	Escapes	Rate
D1 Boundary shift	8	1	0.12
D2 Value / constant drift	9	1	0.11
D3 Omitted check	6	0	0.00
D4 Weakened condition	6	0	0.00
D5 Sequence violation	6	2	0.33
D6 Error-handling drift	6	0	0.00
D7 Comment decoy	6	0	0.00
D8 Unauthorised scope creep	6	6	1.00
D9 Output contract drift	6	3	0.50
All drift	59	13	0.22

Two readings matter. First, testing is a partial defence, not an absent one: 78 percent of injected drift is caught by tests that were never written with drift in mind. Second, the residue is not randomly distributed. Unauthorised scope creep escapes in all six cases, and output contract drift in half, because both concern behaviour the test suite does not assert about. These

are exactly the categories for which a specification-level detector is the only available defence.

IV. DETECTORS UNDER EVALUATION

A. SpecGuard: Retrieval Then Verification

SpecGuard is the detector the benchmark was built to examine. It is deliberately separated into two stages so that failures can be attributed.

Stage 1, rule retrieval, parses the specification into numbered rules and the source tree into chunks at function, method, class and module granularity using the abstract syntax tree, so that a chunk is always a syntactically complete unit. Rules and chunks are embedded into a shared space and ranked by cosine similarity, and the top k chunks per rule are forwarded, with k set to 3 throughout this paper. One refinement proved important: when a retrieved chunk references a module-level constant, the chunk defining that constant is appended to the excerpt. A rule stating that the loan period must be 14 days, judged against the expression `day plus LOAN_PERIOD_DAYS` with the constant's definition out of view, produces a verifier that is not so much wrong as under-informed. Adding constant resolution cut the false-alarm rate on the offline stand-in provider from 0.67 to 0.33.

Stage 2, semantic verification, asks the model about exactly one rule at a time, showing only the retrieved excerpts with their true line numbers, and requires a JSON verdict of COMPLIANT, DRIFT or UNCERTAIN with a confidence, a quoted violated clause, an evidence span and a counterexample. Following [1], the prompt forbids fix suggestions, states that comments and docstrings are claims rather than behaviour, and states that a possible risk is not a violation. An evidence gate is then applied mechanically outside the model: a DRIFT verdict lacking either a quoted clause or a concrete counterexample is downgraded to UNCERTAIN before it reaches the scorer.

B. Baselines

The keyword baseline tokenises each rule and searches all source text, including comments, for its vocabulary and literal values. It is included to establish the floor that any semantic method must clear, and its inclusion of comments is deliberate: it is what makes a comment decoy fool it, which is the point of having it. The whole-file baseline places the entire codebase and the entire specification into a single prompt and asks for a verdict on every rule at once. It represents the naive deployment of a long-context model and isolates the contribution of retrieval, since it differs from SpecGuard in that respect alone.

C. Determinism and Cost

Model responses are cached on a hash of the question, so that only the chunks a case actually modified incur a call: a full 86-case SpecGuard run costs 414 model calls rather than the 1,146 rule-case pairs it covers. Temperature is fixed at zero, retrieval ties are broken by chunk identifier, and every result file records the model identifier used. A deterministic offline provider ships as the default so that the pipeline, the demonstration and the dashboard run with no API key; it is not a model, and the framework prints a warning whenever it is active.

V. EVALUATION PROTOCOL

The positive class is drift. Scoring is stricter than a conventional binary classification in one respect that matters for the trustworthiness question.

- A drift case is a true positive only when the detector flags one of the rules the injection actually targeted. A drift flag that lands on an unrelated rule earns nothing and is tallied separately as a spurious flag, because a detector that is right by accident offers a reviewer nothing to act on.
- D8 cases name no target rule, since no rule authorises the added behaviour, so any drift flag on the case counts as a catch.
- Any drift flag on a negative control is a false alarm. The false-alarm rate is reported alongside precision rather than folded into it.
- UNCERTAIN counts as not flagged, so the evidence gate can only cost recall, never inflate it.

Each miss is additionally attributed. If the chunk containing the injection was never retrieved for the targeted rule, the miss belongs to retrieval; if it was retrieved and the verdict was still COMPLIANT, the miss belongs to reasoning. A detector with no retrieval stage is marked not applicable rather than blamed for a stage it does not have. Stage 1 is independently scored by Recall@k against the gold symbol named in the case manifest.

VI. RESULTS

Results are reported from a single run of each detector over all 86 cases with gpt-4o-mini at temperature zero, supplemented by the retrieval and keyword measurements reproduced for this paper. They are preliminary at n equal to 1; they are reported because their shape, not their precision, is the finding.

A. Stage 1: Retrieval in Isolation

Across 64 rule queries with a gold symbol, the local hashing embedder achieves Recall@1 of 0.27, Recall@3 of 0.78 and Recall@5 of 0.92. At the operating point used throughout, k equal to 3, roughly one injected chunk in five is never shown to the verifier at all, which places a hard ceiling on achievable recall. Fig. 1 breaks Recall@3 down by category.

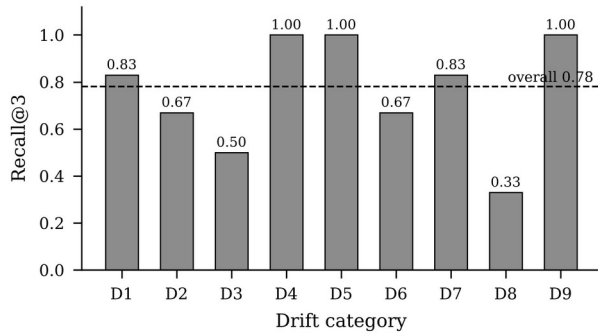


Fig. 1. Stage 1 retrieval quality by drift category at k = 3, against the gold symbol named in each case manifest. Unauthorised scope creep (D8) is the outlier: added behaviour has no rule to be similar to.

The weakest category is D8 at 0.33, and the reason is structural rather than incidental. Retrieval ranks code by similarity to a rule; code that no rule authorises is, by construction, similar to no rule. Reverse-direction drift will need an authorisation pass that iterates over code looking for rules, rather than over rules looking for code. Omitted checks at 0.50 are the second weakest, for the related reason that deleted code leaves nothing behind to retrieve.

B. Detector Comparison

Table IV and Fig. 2 give the end-to-end comparison.

TABLE IV. END-TO-END DETECTOR COMPARISON (N = 1, TEMPERATURE 0)

Detector	P	R	F1	FAR	Unc.	Calls
SpecGuard	0.67	0.58	0.62	0.63	0.06	252
Whole-file	0.91	0.17	0.29	0.04	0.01	86
Keyword	0.33	0.08	0.14	0.37	0.00	0

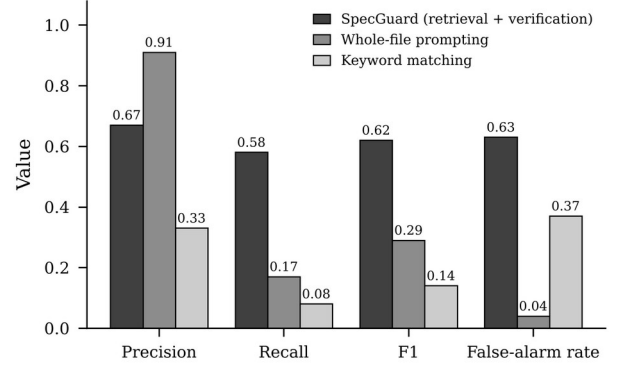


Fig. 2. End-to-end detector comparison. The two LLM-based detectors fail in opposite directions: retrieval buys recall and pays for it in false alarms.

The two LLM-based detectors fail in opposite directions, and the comparison between them is the clearest result in the paper, because they differ only in whether a retrieval stage is present. Retrieval more than triples recall, from 0.17 to 0.58. Showing the model a short, relevant excerpt is what lets it see a violation at all. The same intervention raises the false-alarm rate from 0.04 to 0.63: SpecGuard flags nearly two thirds of the untouched and semantics-preserving controls. Whole-file prompting is quiet and precise and largely blind, missing five of the nine drift categories outright.

The keyword baseline establishes the floor: recall 0.08, false-alarm rate 0.37. It is both deaf and noisy, which is the expected signature of vocabulary matching, and it produced 18 spurious flags on rules no injection touched. Per category it scores zero recall on weakened conditions, sequence violations, error-handling drift, comment decoys and output contract drift, and never exceeds 0.22 on any category.

C. Where Semantic Verification Works

SpecGuard's per-category profile has a consistent shape. It is strongest where a rule states something directly checkable against a value in the code: value and constant drift and weakened conditions are both caught in every case. It is weakest on sequence violations and error-handling drift, at 0.33 each, which require the model to reason about the order in which operations occur and about which exception propagates, rather than to compare a stated literal against a written one. The gradient is from comparison to reasoning, and it is steep.

D. The False Alarms Are Confidently Wrong

The evidence gate works as designed and does not solve the problem. Requiring a quoted clause and a concrete counterexample removes unsupported drift claims. It does not remove wrong ones. Two false alarms from the reported run, both on code that was never modified, illustrate the residue.

- Rule: a new bucket **MUST** start full. The model reported that `bucket_for(...)` yields tokens equal to 0.0. The `dataclass` field is declared `tokens: float =`

BURST_LIMIT. The model misread a default value that was on screen.

- Rule: an allowed request MUST consume exactly 1 token. The model's counterexample was a transition from 1.5 tokens to 0.5 tokens, which is consuming exactly one token, and therefore confirms the rule it was offered as a violation of.

Both arrived with a quoted clause, a well-formed counterexample and a confidence of 0.9, and both passed the evidence gate. This is the same overcorrection Jin and Chen [1] document, reproduced at repository scale and surviving a structural defence against it. It is also the direct argument for the next stage of the work: a counterexample that is executed against the code cannot be fabricated. Had either of these two been run, it would have been discarded in milliseconds.

VII. DISCUSSION

A. Retrieval Buys Recall and Sells Precision

The headline trade-off deserves to be stated plainly, because it is counter-intuitive for anyone who expects retrieval-augmented generation to improve a judge uniformly. A model given the whole codebase answers conservatively and is nearly always right when it does answer, because it mostly does not answer. A model given a short excerpt answers readily, catches three times as much real drift, and invents violations in code that is correct. Retrieval does not make the model a better reader; it makes the model a more willing one. Any deployment of this architecture must budget for that willingness.

B. Evidence Citation Is Necessary and Insufficient

Requiring a drift verdict to quote a clause and exhibit a counterexample eliminates a class of failure, the unsupported assertion, that Jin and Chen identify as the dominant one in their setting. In ours it leaves behind a second class: the supported but false assertion, in which every required artefact is present and the content of one of them is wrong. Syntactic evidence requirements cannot catch this, because the defect is semantic. Only execution can, which is why the two lines of work converge: their Fix-guided Verification Filter executes the model's proposed repair, and our planned adversarial pass must execute the model's proposed counterexample. We cannot adopt their mechanism directly, since asking for a fix is exactly the prompt elaboration their own results show to be harmful, so the counterexample rather than the fix must carry the executable burden.

C. Implications for Deployment

At a false-alarm rate of 0.63 no team would leave this detector enabled in continuous integration, and reporting that number is the point of the exercise. The practical path is not a better prompt but an abstention policy: if confidence scores calibrate against correctness, low-confidence verdicts can be routed to human review and the remainder trusted. The two examples above warn that calibration is not guaranteed, since both wrong verdicts carried confidence 0.9. Measuring calibration is therefore a prerequisite for deployment rather than a refinement of it.

VIII. THREATS TO VALIDITY

Internal validity. Cases are synthetic injections rather than drift observed in the wild, which buys rule-level ground truth at the cost of realism; injections were authored to resemble plausible LLM edits, but the resemblance is a judgement call. Exactly-once matching and test-suite validation guard against

malformed cases, and every negative control is verified to preserve behaviour.

External validity. Three host projects, 40 rules and one language limit generalisation; the three domains were chosen to differ in whether rules concern values, ordering or error behaviour, but they remain small services. The full design targets ten projects and approximately 250 cases.

Construct validity. Reported detector results come from a single run of one model, gpt-4o-mini, at temperature zero. LLM judgement is non-deterministic even so, and no variance is reported here; the differences between detectors are large enough that their direction is unlikely to reverse, but their magnitudes should be read as provisional. Retrieval results use a deterministic local hashing embedder rather than a trained sentence encoder [8], [9], so Stage 1 numbers are a lower bound.

Conclusion validity. Scoring requires a flag to name the targeted rule, which is stricter than case-level detection and depresses recall relative to evaluations that count any flag on a drifted file. We regard the stricter measure as the one that matches how a reviewer would use the output.

IX. CONCLUSION AND FUTURE WORK

SpecDrift-Bench measures what the drift-detection literature has reported around: not how many issues a checker raises, but how many it misses, how many it invents, and in which categories. On 86 validated cases the answer for a retrieval-augmented LLM detector is a recall of 0.58 bought at a false-alarm rate of 0.63, with the remaining false alarms confidently argued rather than unsupported. Twenty-two percent of the injected drift passes the host projects' own tests, and unauthorised scope creep passes them always, so the problem the detector addresses is real even though the detector is not yet trustworthy.

Four extensions are scheduled, each with a slot already present in the framework. An adversarial second pass will execute the counterexample attached to every drift verdict and retain the verdict only if the code behaves as alleged, which directly targets the failure characterised in Section VI-D. A reverse-direction authorisation pass will iterate over code looking for an authorising rule, rather than over rules looking for code, which is the only way to lift D8 retrieval above 0.33. Repeated runs across model scales, with Semcheck added as a third baseline, will replace single-run point estimates with variance. And confidence calibration will be measured to derive the abstention threshold that deployment requires. The benchmark will be expanded to ten host projects and approximately 250 cases over the same taxonomy.

The framework, the case manifests, the host projects and the evaluation harness are available at [12].

ACKNOWLEDGMENT

The authors thank Dr. Vivekanandan S J for his guidance throughout this work, and the School of Computing, SASTRA Deemed to be University, for providing the computing resources used in the experiments.

REFERENCES

- [1] H. Jin and H. Chen, "Are LLMs reliable code reviewers? Systematic overcorrection in requirement conformance judgement," *Automated Software Engineering*, 2026, doi: 10.1007/s10515-026-00638-5.
- [2] ReJot Labs, "Semcheck: semantic checking of implementation against specification using LLMs," 2025. [Online]. Available: <https://github.com/rejot-dev/semcheck>

- [3] GitHub, “Spec Kit: toolkit for spec-driven development,” 2025. [Online]. Available: <https://github.com/github/spec-kit>
- [4] Amazon Web Services, “Kiro: the AI IDE for spec-driven development,” 2025. [Online]. Available: <https://kiro.dev>
- [5] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.
- [6] J. Austin et al., “Program synthesis with large language models,” arXiv:2108.07732, 2021.
- [7] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] N. Reimers and I. Gurevych, “Sentence-BERT: sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [9] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: packed resources for general Chinese embeddings,” arXiv:2309.07597, 2023.
- [10] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, International Organization for Standardization, 2018.
- [11] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: can language models resolve real-world GitHub issues?” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- [12] K. Nandesh, S. Soundar, and N. Sudharsan, “SpecDrift-Bench,” 2026. [Online]. Available: <https://github.com/nandeshkanagaraju/specdrift-bench>